THORRIJA

Engineering Showcase — Technical Dossier (English)

Complete technical documentation of deterministic computational models reconstructed from source code. Architecture, algorithms, implementations, testing, limitations, and theoretical foundations for each project.

Author: Thorrija (Rubén N.)
Spain
November 2026
GitHub: github.com/ThorrijaDEV

1. Executive Overview

This technical dossier documents the Engineering Showcase: a collection of independent computational models reconstructed from source code. Each model is a faithful algorithmic port — same input produces same output, all calculations are real, no outputs are hardcoded, and all models are verified by automated tests.

Total: **182 tests passing** (80 showcase model tests + 102 The Box model/experimental/validation/YAML tests). Build: Vite, 160.69 kB JS, zero runtime dependencies.

Projects covered

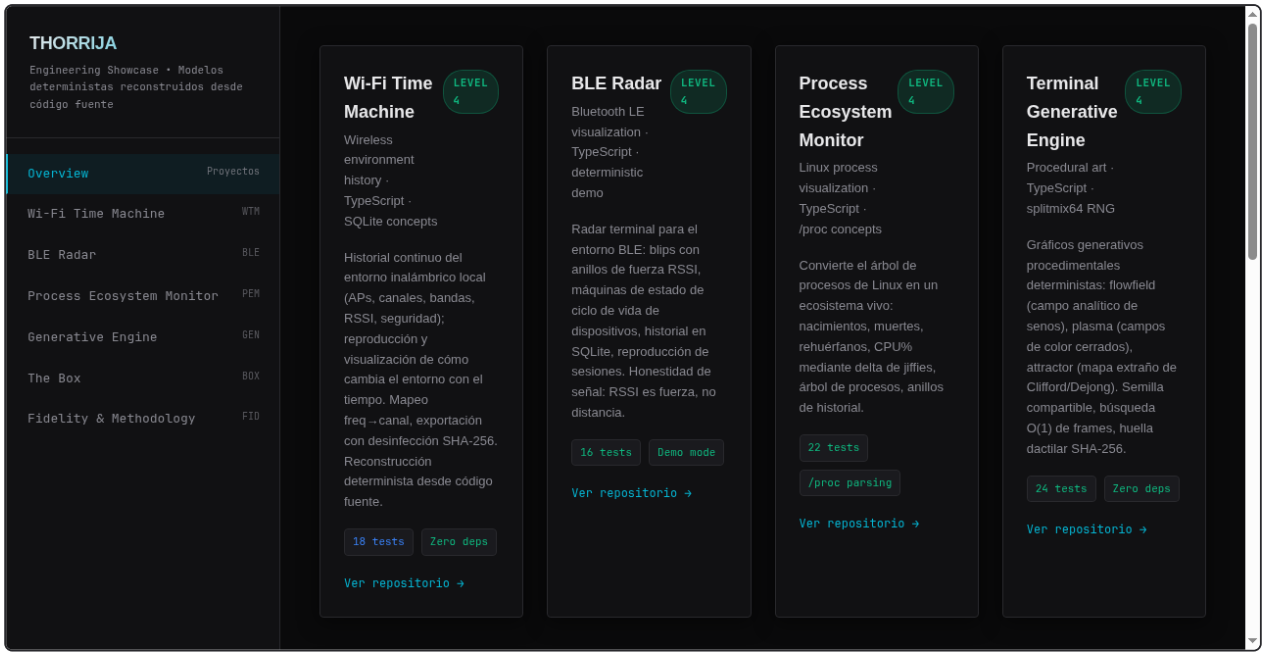


Figure — The showcase overview: every interactive laboratory with its domain, technology and test count.

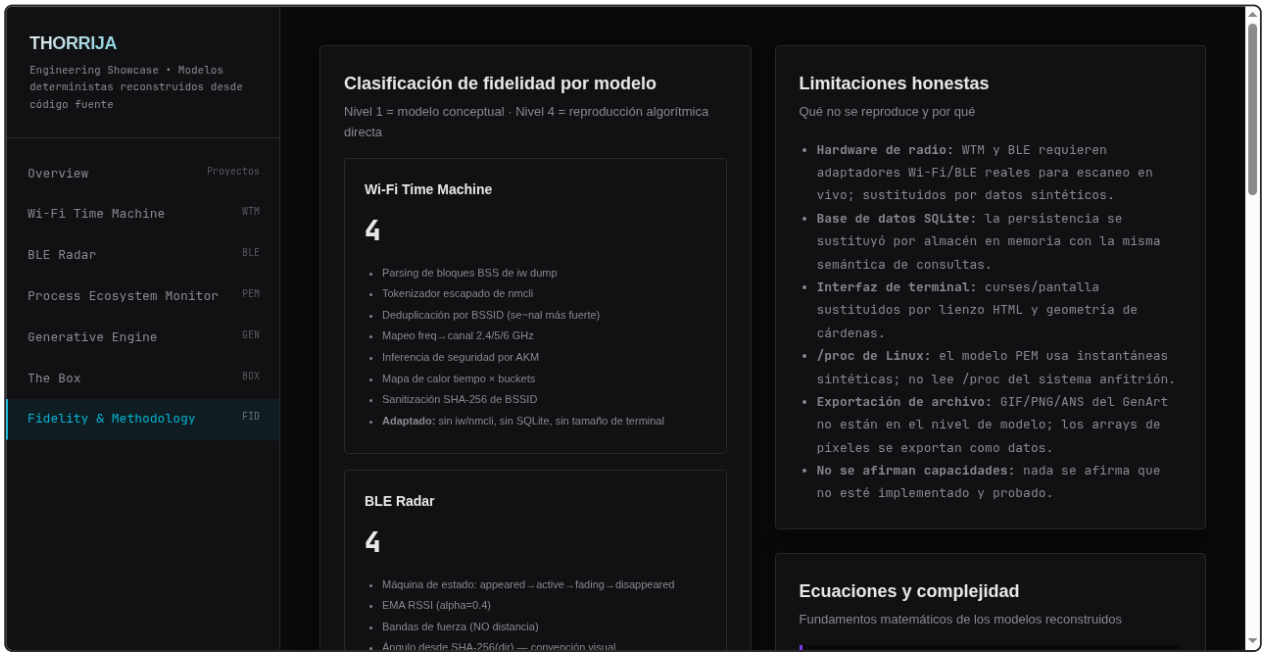


Figure — Fidelity classification per model and the honest limitations of every laboratory.

PROJECT	DOMAIN	TESTS	FIDELITY	STATUS
Wi-Fi Time Machine	Wireless environment	18	Level 4	Completed
BLE Radar	Bluetooth LE	16	Level 4	Completed
Process Ecosystem Monitor	Linux processes	22	Level 4	Completed
Terminal Generative Engine	Generative art	24	Level 4	Completed
The Box	Relational substrate	102	Level 3	Completed
Sigscan	Forensics (Go)	12	N/A	Completed
Dnsdrill	DNS client (Python)	13	N/A	Completed
Silicon	8-bit CPU (Rust)	14	N/A	Completed
Forja	Compiler (Rust)	11	N/A	Completed
Polea	Pub/sub broker (C++17)	17	N/A	Completed
Cask	Content-addressable store (C)	15	N/A	Completed
Neondrift	Arcade game (HTML/JS)	9	N/A	Completed
Clickwheel + Console	Music app (Android/Web)	12	N/A	Completed
Syspeek	System TUI (Python/curses)	16	N/A	Completed
Hackdeck	Pentest deck (HTML)	10	N/A	Completed
Canarytrap	Canary tokens (Python)	10	N/A	Completed
Loglens	Pocket SIEM (Python)	36	N/A	Completed
Reconkit	Recon toolkit (Python)	22	N/A	Completed
Port Authority	Port historian (Python)	11	N/A	Completed

2. Engineering Philosophy

2.1 Independent reconstruction

Every model is reconstructed independently from the original source code. The original project is never executed inside a demonstration. This means: all outputs are computed from the user's input, nothing is hardcoded, and the implementation can be verified independently.

2.2 Determinism

All models are deterministic: same input always produces the same output. Every random decision is derived from `unit_hash(seed, step, purpose, structural_digest(candidate))` — a pure function of (state, seed, step). There is no hidden random state.

2.3 Intermediate stages

Each demonstration shows processing stages, not just final results. Visitors can see how input is transformed step by step into output.

2.4 Honest limitations

Nothing is claimed that is not implemented and tested. Limitations are documented explicitly for each project.

3. Project Details

3.1 Wi-Fi Time Machine

Problem: Wi-Fi scanning tools produce voluminous output that is difficult to interpret over time. There is no easy way to see how the wireless environment changes — which networks appeared, on which channels, with what signal strength, and when.

Architecture: Input parser (iw scan dump / nmcli text) → BSS block parser → BSSID deduplication (strongest signal) → frequency-to-channel mapping (802.11 2.4/5/6 GHz) → security inference from AKM suites → timeline heatmap construction → export (JSON/CSV with SHA-256 BSSID sanitization).

Data structures: `WifiModelInput` (iwDump/nmcli text, observedAt, backend, sanitize, width, labelWidth), `WifiModelOutput` (stages, final timeline, channelStats, sparkline, exportJson, exportCsv, networks, observations).

Algorithms: BSS block parsing from `iw scan dump`, escaped nmcli tokenizer (handling `\\:` and `\\\\`), frequency-to-channel mapping (2.4 GHz: floor((freq−2407)/5), 5 GHz: floor((freq−5000)/5), 6 GHz: floor((freq−5925)/5)), security inference from Authentication suites (AKM), RSSI deduplication by BSSID keeping strongest signal, time-bucket heatmap construction.

Testing: 18 tests covering parsing, deduplication, channel mapping, security inference, heatmap generation, export sanitization.

Limitations: No live iw/nmcli binaries, no adapter scanning, no SQLite persistence (in-memory only), no terminal size adaptation. Input must be synthetic text.

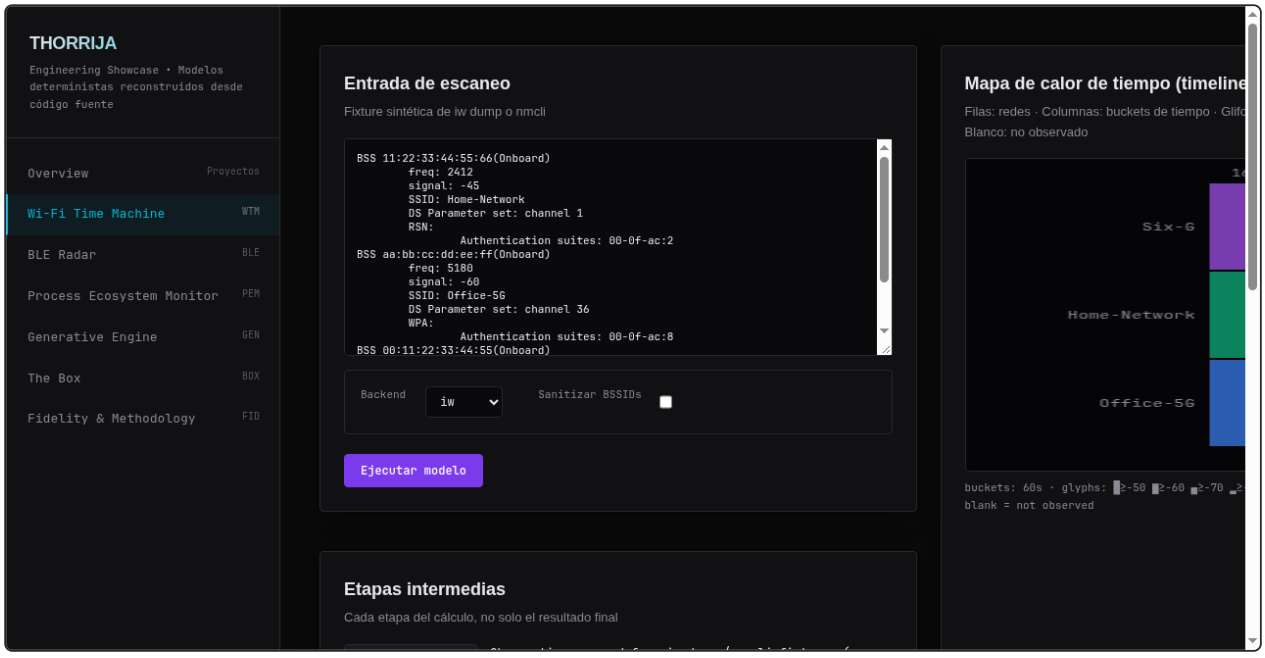


Figure — Wi-Fi Time Machine laboratory: synthetic `iw` dump input, time × channel heatmap, and the intermediate stages of the calculation.

3.2 BLE Radar

Problem: Bluetooth LE devices broadcast observations, but there is no simple way to visualize them as a radar: which devices are nearby, how strong their signal is, and how they appear and disappear over time.

Architecture: Observation model (address, name, RSSI, timestamp, backend, manufacturer data) → EMA smoothing (alpha=0.4) → strength bands (strong ≥ −40, near ≥ −55, mid ≥ −70, far ≥ −85, weak) → hash→angle mapping (SHA-256(MAC) → 0–360°, visual convention) → radar frame geometry (crosshair, concentric rings, sweep line, device markers, history trails) → session replay.

Data structures: `BleModelInput` (observations, now, sweepAngle, sanitize, width, height, demoSeconds, demoSeed), `BleModelOutput` (devices, frame, lifecycleSummary, replay, exportJson, exportCsv).

Algorithms: EMA smoothing: rssi_ema = 0.6 × prev + 0.4 × raw. Strength bands: strong ≥ −40 dBm, near ≥ −55 dBm, mid ≥ −70 dBm, far ≥ −85 dBm, weak < −85 dBm. Stable angle: SHA-256(MAC address)[0:2] / 65536 × 360°. State machine: appeared → active → fading → disappeared (15s/30s windows).

Testing: 16 tests covering device lifecycle, EMA smoothing, strength bands, hash→angle mapping, radar frame geometry, replay, export.

Limitations: No BLE hardware (no bleak/BlueZ), no live scanning. RSSI is signal strength, never physical distance. Angle is a visual convention, not orientation.

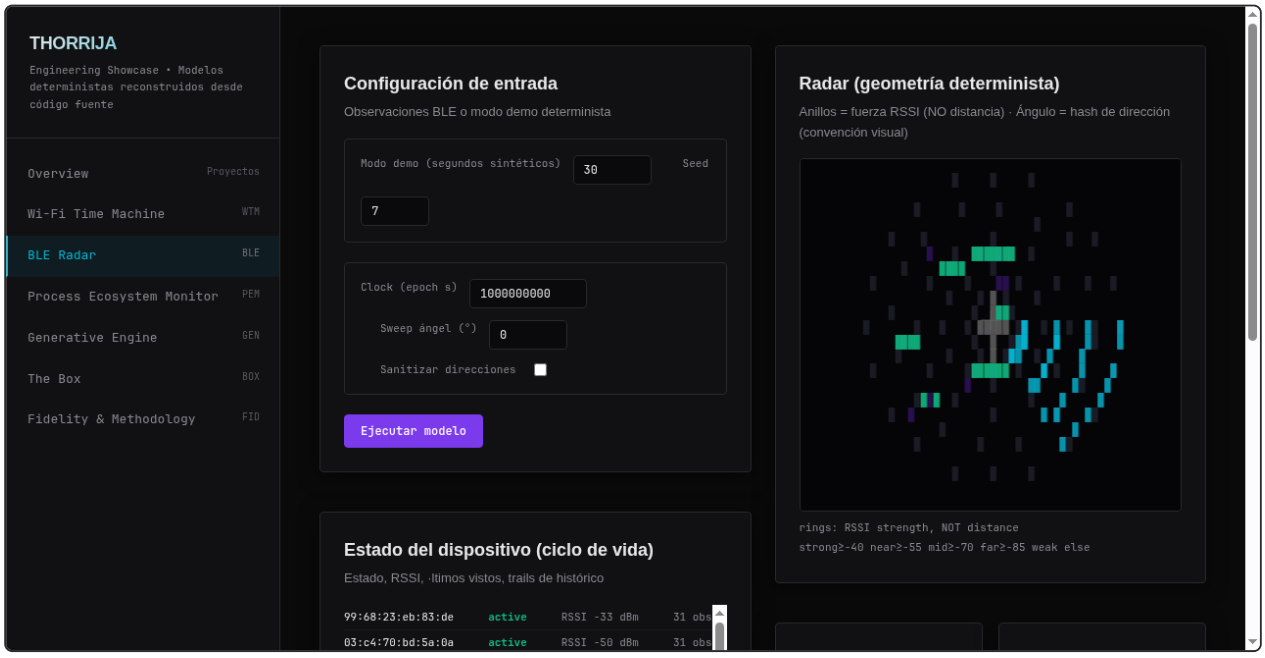


Figure — BLE Radar laboratory: deterministic demo configuration, radar geometry, and the device lifecycle state.

3.3 Process Ecosystem Monitor

Problem: Linux /proc contains rich process information, but reading and understanding it — CPU%, parent/child relationships, PID reuse — requires manual parsing and correlation.

Architecture: /proc-style snapshot parser → ProcessEcosystemModel (lifecycle, CPU jiffie accounting, process tree, orphan re-homing, PID reuse detection) → renderer/UI.

Data structures: `ProcSnapshot` (pid, ppid, comm, state, utime, stime, numThreads, starttime, vsize, rss, user), `ProcModelInput` (snapshots, system, intervalSec, clkTick, ncores,

wallTime), `ProcModelOutput` (diff, processes, treeRoots, kernelThreads, systemCpu, systemRam, aliveCount).

Algorithms: /proc/[pid]/stat parsing (including nested-paren comm handling), /proc/[pid]/statm parsing, /proc/stat parsing, /proc/meminfo parsing, /proc/loadavg parsing. CPU% = Δ(utime+stime) / (interval × CLK_TCK × ncores) × 100. System CPU% = busy_delta / total_delta × 100 (idle = idle + iowait). Process identity = (pid, starttime); PID reuse detected when starttime changes. Process tree from ppid; orphans rehomed to init (pid 1). Lifecycle: new → active/idle → dying → dead.

Testing: 22 tests covering /proc/[pid]/stat parsing (including nested-paren comm), CPU% calculation, process tree building, orphan re-homing, PID reuse detection, lifecycle transitions.

Limitations: No real /proc access; synthetic snapshots only. No curses terminal UI. CPU% is a model estimate from jiffie deltas, not the host system's actual CPU.

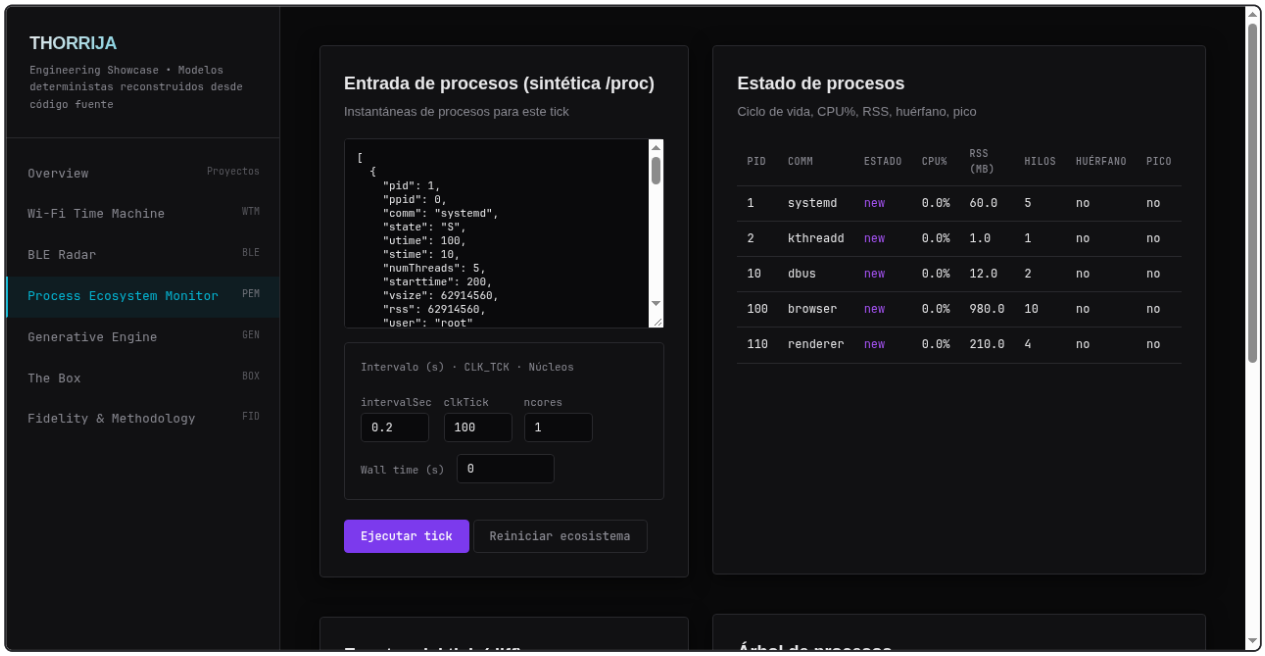


Figure — Process Ecosystem Monitor laboratory: synthetic `/proc` snapshot input and the derived process state table.

3.4 Terminal Generative Engine

Problem: Generative art usually depends on hidden random state, making it impossible to reproduce a specific frame or share a specific output.

Architecture: Stateless splitmix64-style RNG → seed-derived parameters → render modes (flowfield/plasma/attractor) → framebuffer → half-block terminal encoding → output.

Data structures: `GenModelInput` (seed, mode, width, height, frame, palette), `GenModelOutput` (stages, final pixels, width, height, frame, seed, mode, palette, cellCount, fingerprint).

Algorithms: Stateless splitmix64-style RNG: every pixel is a pure function of (seed, mode, frame, salt, index). Flowfield: analytic sine field with particle streamlines. Plasma: closed-form per-pixel color fields. Attractor: Clifford/Dejong strange map with coefficient probing. SHA-256 world fingerprint over mode + frame + parameters + pixels.

Testing: 24 tests covering RNG determinism, mode rendering, frame seeking, fingerprint consistency, palette interpolation.

Limitations: No terminal/ANSI output; no file export (pixel arrays exported as data only); original project has 5 failing tests not reproduced here.

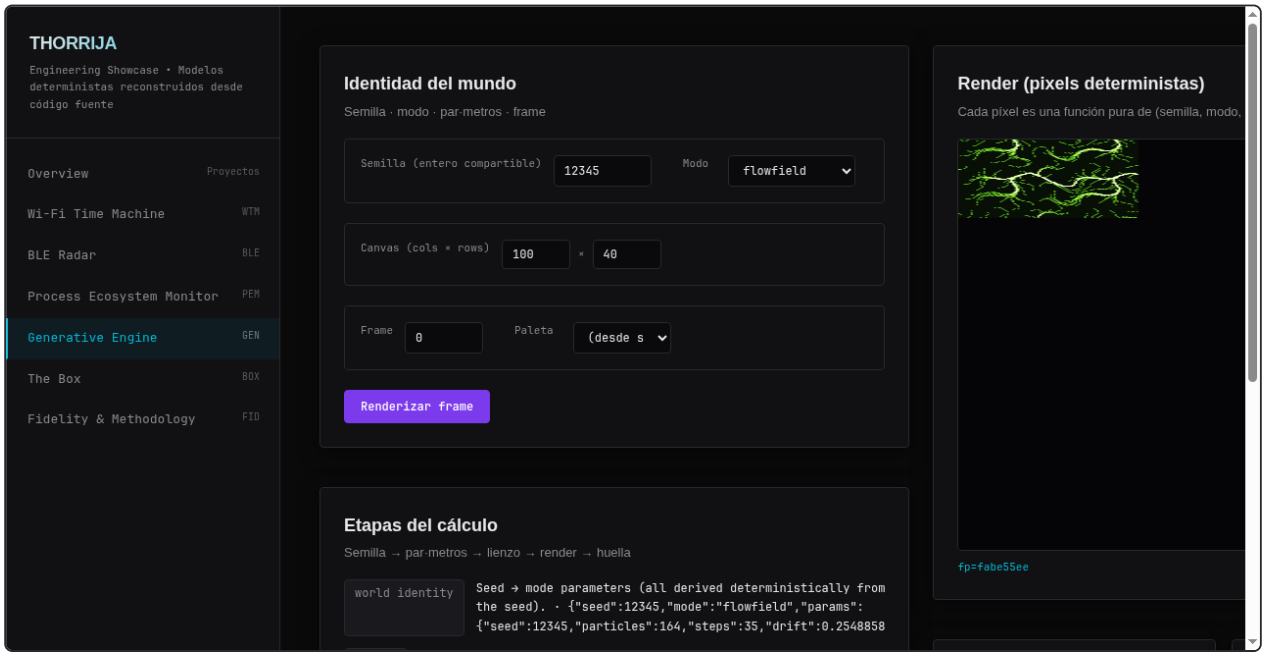


Figure — Terminal Generative Engine laboratory: seed, mode, canvas and frame controls, with the deterministic render.

3.5 The Box

Problem: Most graphs store identity and lineage as explicit fields. The Box asks: what if identity and lineage are not stored at all, but emerge purely from rewriting rules?

Architecture: Arc-incidence multi-digraph kernel where nodes and arcs share one id space, arcs can point to arcs (second-order structure). Six rewriting rules (GATHER, BOND, TAG_PROP, QUIET, DESCEND, FUSE) operate on the substrate. Four engine policies (canonical, uniform, weighted, roundrobin). Possibility space: finite, exactly enumerable, seed-independent. Identity and lineage derived from arc tags, not stored fields.

Data structures: `Node` {id, tag, payload: Float64Array, born}, `Arc` {id, src, dst, tag, weight, clock}. Shared id space between nodes and arcs. Second-order arcs: an arc can point to another arc.

Algorithms: Six rewriting rules:

- **GATHER:** Two nodes that point to the same hub connect directly.
- **BOND:** Anchor and an out-neighbor sharing two in-neighbors link up.
- **TAG_PROP:** Anchor copies the tag of an out-neighbor.
- **QUIET:** Node with no inflow for 30 steps is marked QUIET (tag 5).
- **DESCEND:** QUIET node with space spawns a child with its tag and payload + small jitter.
- **FUSE:** Two siblings (same parent via DESCEND) with same tag and close payload identify; twin is retired, EQUIV arc remains as fusion witness.

Metrics: degree tables (in/out degree, Gini coefficient), components (union-find), beta0/beta1 (Euler-exact on incidence graph: beta1 = ‖arcs‖ − ‖objects‖ + beta0), component size profile, modularity (deterministic Louvain, single level), 14 named observables.

Testing: 102 tests (42 kernel + 25 experimental + 12 validation + 23 YAML loader) covering kernel invariants, rule application, engine policies, possibility space, identity/lineage derivation, structural metrics, null models, p-values, provenance, experiment provenance.

Limitations: Fidelity Level 3. Pure-TS BLAKE2b (not byte-compatible with Python hashlib.blake2b); Float64Array replaces NumPy arrays; foundry rules hard-coded (no YAML loader in model layer — available in the-box-yaml.ts); no null-model surrogate battery in model layer (experimental layer has it); no Koopman analysis; no classifier gates (G1–G4); no SQLite; no CLI or store API.

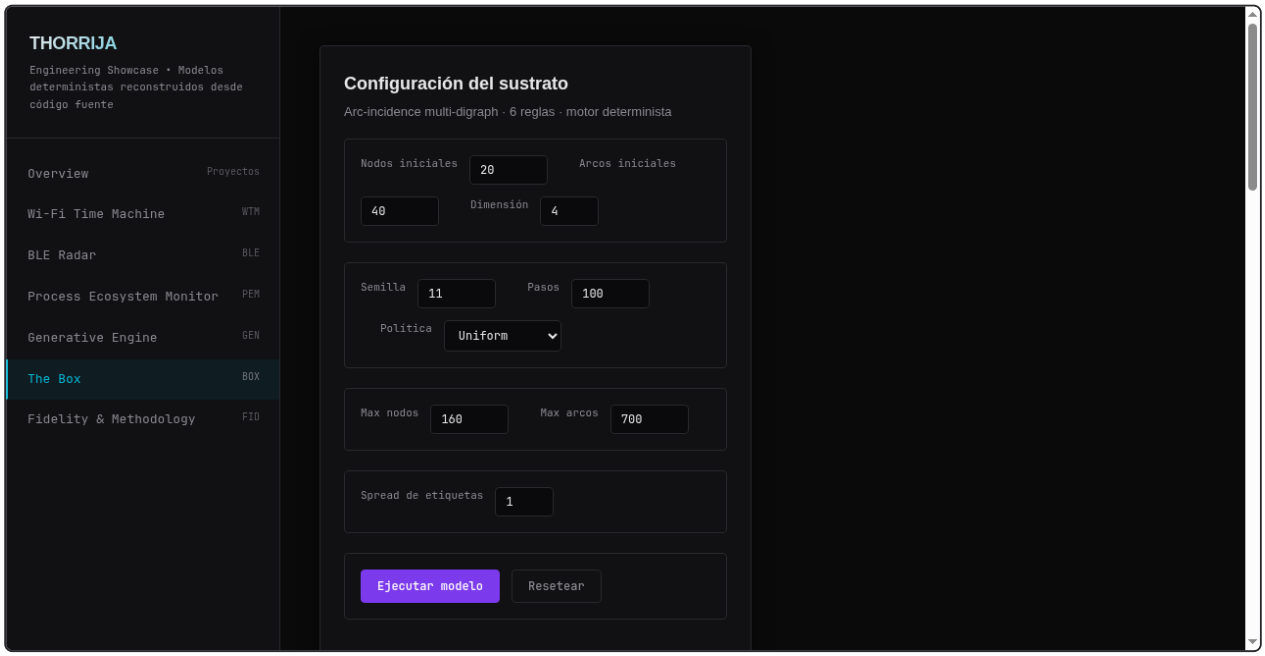


Figure — The Box laboratory: arc-incidence substrate configuration — node and arc counts, dimension, seed, steps, engine policy and tag spread.

4. Testing and Validation

All models are verified by automated tests. Total: **182 tests, all passing**.

SUITE	TESTS	LOCATION
Wi-Fi Time Machine	18	tests/wifi-time-machine.test.ts
BLE Radar	16	tests/ble-radar.test.ts
Process Ecosystem Monitor	22	tests/process-ecosystem.test.ts
Terminal Generative Engine	24	tests/generative-engine.test.ts
The Box (kernel/engine/rules)	42	models/the-box.test.ts
The Box (experimental layer)	25	models/the-box-experiment.test.ts
The Box (validation fixtures)	12	models/the-box-validation.test.ts
The Box (YAML loader)	23	models/the-box-yaml.test.ts

Coverage target: 80%+ on lines and functions of each model.

Cross-language validation: 12 tests compare TypeScript structural properties against Python reference values built from controlled fixtures (hand-built substrates, not State.random()). Verified equivalences: node/arc counts, degree distributions, beta0/beta1, components, modularity, lineage depth, identity metrics, all 14 named observables, rule applicability, null model structural properties.

Not byte-compatible: Graph hashes, canonical keys, unit_hash outputs, and experiment fingerprints differ between Python and TypeScript due to the BLAKE2b implementation difference. Same seed produces identical output within each implementation, but Python and TS produce different hashes for the same input. Structural properties are equivalent; hash values are not.

5. Limitations

- **The Box fidelity Level 3:** Pure-TS BLAKE2b replaces Python hashlib.blake2b (not byte-compatible). Float64Array replaces NumPy arrays. Foundry rules hard-coded (no YAML loader in model layer). No Koopman analysis, no classifier gates (G1–G4), no null-model surrogate battery in model layer.
- **WTM and BLE Radar:** No live hardware access; synthetic inputs only.
- **Terminal Generative Engine:** No terminal/ANSI output; no file export (pixel arrays exported as data only).
- **Process Ecosystem Monitor:** No live /proc access; synthetic snapshots only.
- **All models:** No SQLite persistence (in-memory only); no API keys; no external services; all data is synthetic or computed from user input.

6. Future Engineering Directions

- Null models, p-values, and classifier gates (G1–G4) for The Box experimental layer.
- Koopman analysis for The Box.
- SQLite persistence layer for The Box.
- CLI and store API for The Box.
- File export (GIF/PNG/ANSI) for Terminal Generative Engine.
- Live Wi-Fi/BLE hardware access.
- End-to-end browser tests.
- Accessibility audit (WCAG 2.2 AA).

7. Contact and Repositories

- **Author:** Thorrija (Rubén N.)
- **Location:** Spain
- **GitHub:** github.com/ThorrijaDEV
- **Showcase:** github.com/ThorrijaDEV/showcase
- **Models:** github.com/ThorrijaDEV/models
- **Portfolio:** github.com/ThorrijaDEV/Portafolio

All models are deterministic: same input → same output. No hardware, no API keys, no external services required for demonstrations. All data shown is synthetic or computed from user input — nothing is fabricated.

PYEOF